

AIKernel Core Specification: Chat-Turn HashChain Governance

決定論的チャットターン同一性とリプレイ可能な因果トレースモデル

Takuya Sogawa

2026-06-01

著者: Takuya Sogawa

所属: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

Version: v0.1.0

Date: 2026-06-01

DOI: 10.5281/zenodo.20471345

License: CC BY 4.0

概要

本仕様は、チャットベースの対話履歴を、因果的に連結され、改ざん検知可能で、リプレイ可能な意味的実行記録へ変換するための AIKernel Core 仕様として、Chat-Turn HashChain Governance を定義する。

AIKernel では、Phase-1 において知識スナップショット基盤としての ROM、推論前の受理判定を行う Pre-Inference Governance、推論軌跡を監視する Trajectory Governance が定義された。Phase-2 では、Semantic State、Semantic Transition、Observability、および Semantic Runtime Theory が導入された。しかし、チャット履歴における各ターン単位の因果的同一性を Core Contract として扱う仕様は、まだ明示的には定義されていなかった。

本仕様は、このギャップを埋めるため、各チャットターンを、正準化され、前ターンのハッシュと結合され、必要に応じて署名された、Append-only なガバナンスノードとして扱う。

本仕様の主な貢献は、チャットターンの正準化、ハッシュチェーン構築、署名検証、Fail-Closed 検証、Quarantine、決定論的リプレイ、VFS/ROM 永続化、および Cryptographic Agility を、AIKernel Core の観点から統一的に定義する点にある。本仕様は特定の暗号アルゴリズムに依存せず、ISemanticHasher および IChatTurnSignatureProvider などの Provider 抽象を通じて暗号アルゴリズムを差し替え可能にする。

英語版を正本とし、日本語版は companion translation として添付する。

1. Overview

AIKernel は、AI の実行を単なるプロンプト応答ではなく、統治された意味的軌跡として扱う。意味的軌跡は、観測可能で、監査可能で、再現可能であり、契約境界によって制約されていなければならない。

しかし、チャットベースのシステムにおいて、基本単位は通常「ターン」である。通常のチャットログは、因果的同一性、不変性、署名による出所証明、リプレイ安全性、フォーク検出をそれ自体では保証しない。

Chat-Turn HashChain Governance は、各チャットターンを正準化済みかつハッシュ連結されたガバナンスノードとして定義する。ターンは、正準化表現、前回ハッシュ、現在ハッシュ、任意の署名、およびガバナンスポリシー検証をすべて通過した場合にのみ、Kernel の Valid State に結合される。

本仕様の目的は、非決定的な LLM 対話履歴を、決定論的に検証可能でリプレイ可能な HashChain へ変換することである。このモデルでは、Semantic Trajectory は単なる会話記録ではない。各ターンで完全性を検証可能な、因果的に連結された実行記録である。

Minimal Example of a Governed Chat-Turn HashChain

Role	PrevHash	CanonicalContent	Hash	State
User	0x00000000	T1	0x1619E3AA	Valid
Assistant	0x1619E3AA	T2	0x60192CAF	Valid
User	0x60192CAF	T3	0x07743DFF	Valid

各ターンは直前の有効ターンのハッシュを取り込む。したがって、過去のターンが変更されると、それ以降のチェーン全体が不整合となる。

2. Canonicalization Model（正準化モデル）

信頼できる HashChain の前提は正準化である。同じ意味内容を持つターンは同じバイト列へ変換されるべきであり、実質的に異なるターンが同じ正準化ペイロードへ潰れてはならない。

2.1 Canonical Form of a Chat Turn

チャットターン T_n は、少なくとも以下のフィールドを含む厳密なデータ構造として定義される。

Field	Meaning
Actor	発話主体または発生源。例: User, Assistant, System, Tool
Body	テキストペイロード、ツール結果、構造化メッセージ等
Timestamp	ISO 8601 UTC など、厳密に固定された時刻表現
TurnKind	通常発話、ツール呼び出し、ツール結果、システムイベント等
Metadata	契約上の正しさに必要な最小限のメタデータ

Metadata は最小限でなければならない。UI 状態、ローカルキャッシュ識別子、メモリ使用量、処理時間、その他の非決定的な実行時情報は、正準ハッシュペイロードに含めてはならない。

2.2 Deterministic Serialization Rules

正準化関数 f_{canon} は、以下の決定論的規則に従って T_n をシリアルライズする。

1. Stable ordering: オブジェクトのプロパティを辞書式順序、または仕様で定義された固定順序で整列する。
2. Whitespace normalization: 意図的でない前後空白や改行差異を正規化する。ただし、コードブロックや引用内の意味ある空白は保持する。
3. Encoding: 仕様で定義されたエンコーディング、たとえば UTF-8 へ変換する。

4. Optional field handling: 省略された optional field、明示的な null、空文字列を暗黙に同一視しない。
5. Culture-invariant formatting: 日付、数値、小数点、大小文字変換はロケール非依存で処理する。

2.3 Forbidden Fields

正準化ペイロードには、ハッシュ計算後に変化する可能性のある情報、または実行環境ごとに異なる情報を含めてはならない。

- UI の展開状態、スクロール位置、テーマ、表示領域
- 処理時間、メモリ使用量
- 一時的なローカルファイルパス
- 非決定論的な乱数 ID
- Provider 内部診断情報の順序
- 契約上定義されていないキャッシュキー
- ログ出力順序に依存する情報

目的は、偶発的な表示状態や実行状態ではなく、統治対象となる意味的コンテキストをハッシュ化することである。

3. HashChain Definition (ハッシュチェーン定義)

正準化後、各ターンは直前の有効ハッシュと結合されてハッシュ化される。これにより、単なる追記リストではなく、因果的なチェーンが形成される。

3.1 Formal Definition

ターン n のハッシュ H_n は以下のように定義される。

$$H_n = \text{Hash}(\text{Canonicalize}(T_n) \parallel H_{n-1})$$

ここで、 $\parallel$ はバイト列結合を表す。初期値 H_0 は Kernel によって定義される Root Hash である。最小実装では 0x00000000 のような固定値を使ってもよいが、実運用ではセッション境界、Kernel identity、ROM root、または replay boundary から明示的に導出することが望ましい。

3.2 Properties

Immutability (不変性) 過去のターン T_{n-k} が 1 ビットでも変更されると、そのハッシュが変化する。後続のすべてのハッシュは、変更された前段に依存するため無効になる。

Causal Dependency (因果的依存性) H_n は、ターン n までの完全な因果的会話状態にコミットする。現在の Semantic Context は、最新メッセージだけでなく、そこに至る有効チェーン全体によって定義される。

Irreversibility (不可逆性) ハッシュから元のメッセージや文脈を復元することはできない。ハッシュは内容そのものではなく、因果状態の識別子である。

Compression of Semantic Entropy (意味的エントロピーの圧縮) 大規模な対話履歴は、決定論的な固定長状態識別子へ圧縮される。この識別子は、Replay、Audit、Fork Detection、Recovery の基礎となる。

4. Signature Model（署名モデル）

HashChain は改ざんを検出するが、それ自体では「誰が生成または承認したか」を証明しない。エンタープライズ利用、マルチプロバイダ実行、Federation、Phase-3 Trust Layer 拡張では、選択されたターンまたは境界ターンに署名を付与することが推奨される。

4.1 Formal Definition

ターン n の署名 $Sign_n$ は以下のように定義される。

$$Sign_n = Sign(H_n, K_{private})$$

ここで、 $K_{private}$ は Actor、Provider、Kernel、Organization、または署名権限主体の秘密鍵である。

4.2 Behavior and Rules

Provider Identity 署名は、ターンを生成または承認した Provider、System、Tool、Kernel、Organization、Actor と紐付けられる。

Verification Rules ターンを受理する際、Kernel は以下を検証する。

1. 正準化表現が仕様に従っていること。
2. PrevHash が現在のチェーン末尾と一致すること。
3. Hash が再計算結果と一致すること。
4. 署名が存在する場合、許可された公開鍵で検証できること。
5. アルゴリズムタグが設定済み Provider によってサポートされていること。
6. ガバナンスポリシーがターンの結合を許可すること。

Fail-Closed Behavior ハッシュ検証、署名検証、正準化、アルゴリズムルーティングのいずれかに失敗した場合、そのターンは Kernel の Valid State に結合してはならない。Kernel は Fail-Closed し、必要に応じて当該ターンを Quarantine へ隔離する。

ReplayLog Integration 有効な署名付きターンは ReplayLog 互換ストレージへ永続化され、後日の検証、監査、決定論的再構築に利用される。

5. Governance Rules（統治ルール）

AIKernel は、生成されたチャットターンを無条件に受け入れない。各ターンは、意味状態の一部になる前に HashChain Governance によって検査される。

5.1 Validation Pipeline

1. 入力ターン T_n を受け取る。
2. $Canonicalize(T_n)$ を計算する。
3. 現在の末尾ハッシュ H_{n-1} と結合し、 H_n を再計算する。
4. 記録済みハッシュと再計算ハッシュを比較する。
5. 署名が存在する場合は検証する。
6. ガバナンスポリシーを適用する。
7. すべての検査に成功した場合のみ、Valid State として受理する。

5.2 On-the-Fly Verification

検証は、会話終了後ではなく、各ターン発生時に実行される。これにより、破損、注入、不正署名、ポリシー拒否されたターンが後続コンテキストへ混入することを防ぐ。

5.3 Quarantine Behavior and Failure Modes

検証に失敗したターンは Quarantine 領域へ隔離される。Quarantine されたターンは、Semantic Context、ReplayLog、Execution Pipeline、Trajectory Evaluation に結合されてはならない。

Failure Mode	Description	Kernel Action
hash.mismatch	記録ハッシュと再計算ハッシュが一致しない	Fail-Closed
prev_hash.mismatch	PrevHash が現在の末尾と一致しない	Fail-Closed
signature.invalid	署名を検証できない	Fail-Closed
algorithm.unsupported	アルゴリズムタグに対応する Provider が存在しない	Fail-Closed
canonicalization.invalid	正準化規則に違反	Reject
policy.denied	ガバナンスポリシーがターンを拒否	Quarantine
injection.detected	Prompt Injection または危険ペイロードを検出	Quarantine

5.4 Recovery Semantics

Fail-Closed により実行が停止した場合、Kernel は最後に検証済みのハッシュ H_{n-1} までロールバックできる。その状態から再開するか、管理者が Quarantine されたターンを調査したうえで、必要に応じて再結合操作を行う。

6. Deterministic Replay (決定論的リプレイ)

保護されたチャットターン HashChain は、ReplayLog v2 の基盤となる。

6.1 Rollback to Any Valid Turn

有効ノードのハッシュ H_k を指定すると、Kernel は検証済みチェーンノードのみをリプレイし、そのターンまでのコンテキストを再構築できる。

6.2 Reconstructing Semantic State

リプレイは、未来の LLM 呼び出しの非決定性ではなく、受理済みチェーンから意味状態を復元する。これにより、過去状態の再構築とモデル確率性を分離できる。

6.3 Bounded Trajectory Reconstruction

Semantic Trajectory は、有界で検証可能な遷移列になる。各受理済み状態遷移は、ハッシュ連結され、ポリシー検証され、必要に応じて署名されたターンへ追跡できる。

7. Runtime Interfaces (ランタイムインターフェース)

以下は AIKernel.NET における最小実装境界を示す。

```
namespace AIKernel.Abstractions.Governance.ChatChain;

/// <summary>
/// チャットターンを決定論的なバイト列または文字列表現へ正準化する。
/// </summary>
public interface IChatTurnCanonicalizer
{
    string Canonicalize(IChatTurn turn);
}

/// <summary>
/// 正準化済みコンテンツと前回ハッシュから現在状態ハッシュを計算する。
/// </summary>
public interface ISemanticHasher
{
    string ComputeHash(string canonicalContent, string previousHash);
}

/// <summary>
/// チャットターンのハッシュに対する暗号的署名を提供する。
/// </summary>
public interface IChatTurnSignatureProvider
{
    Task<string> SignAsync(
        string hash,
        CancellationToken cancellationToken = default);
    Task<bool> VerifyAsync(
        string hash,
        string signature,
        CancellationToken cancellationToken = default);
}

/// <summary>
/// 因果的同一性、ハッシュ連続性、署名、ポリシー受理を検証する。
/// </summary>
public interface IChatTurnChainVerifier
{
    Result VerifyChain(IEnumerable<IHashChainNode> turns);
    Result VerifyNextTurn(
        IHashChainNode nextTurn,
        string currentTailHash);
}
```

ISemanticHasher および IChatTurnSignatureProvider は意図的にアルゴリズム非依存である。具体実装は、設定ポリシーが許可する範囲で、SHA-256、SHA-3、Ed25519、ECDSA P-256、ML-DSA、または将来のアルゴリズムを使用できる。

8. Storage Model (ストレージモデル)

HashChain の永続化は AIKernel VFS および ROM ストレージと統合される。ストレージモデルは単なる永続化の詳細ではなく、ガバナンス境界そのものである。保存されたチャットターンは、メモ

リ外へ出た後も、因果的に連結され、検証可能で、リプレイ可能でなければならない。

8.1 VFS 統合と Append-only の強制

ハッシュチェーン化された対話記録は、AIKernel の VFS レイヤーを介して永続化される。チャットターン保存を担う VFS Provider は、既に受理済みのレコードに対する破壊的な上書き書き換えを拒否しなければならない。許可されるのは、現在の有効チェーン末尾を拡張する append 操作のみである。

準拠するストレージ Provider は、以下の規則を強制する。

- HashChain ノードは JSON、ROM Markdown、またはその他の正準化可能な保存形式としてシリアル化される。
- 各保存ノードは PrevHash、Hash、必要に応じて Signature を含まなければならない。
- 既に受理済みのレコードは、インプレースで上書きされてはならない。
- 新規ターンは、PrevHash が現在の有効チェーン末尾ハッシュと一致する場合にのみ append できる。
- VFS の write 操作は、永続化前に write-time verification を実行すべきである。
- ROM/VFS メタデータには、アルゴリズムタグ、Provider identity、検証状態、quarantine state、replay boundary を含めることができる。

このモデルにおいて、append-only は単なるストレージ上の利便性ではなく、ガバナンス特性である。VFS Provider が append-only 永続化を強制できない場合、外部の不変ログまたは同等の強制層が存在しない限り、Kernel はその Provider を信頼済み HashChain ストレージとして不適格と見なさなければならない。

8.2 ROM Identity への定着

統治された対話がクローズ、確定、または知識資産としてエクスポートされる時点で、有効チェーン全体は単一の不変 ROM Identity として凍結できる。この時点で、チャット履歴は可変のトランスクリプトではなく、永続的な Provenance を持つ封印済み知識スナップショットとなる。

この定着ステップは、Chat-Turn HashChain Governance を AIKernel の ROM モデルへ接続する。

- チェーンの root と final tail hash は因果境界を定義する。
- 正準化されたターン列は意味内容を定義する。
- 署名メタデータは provenance と trust context を定義する。
- ROM Identity は対話を不変の知識スナップショットとして凍結する。

これにより生成された資産は、可変のチャットログを信頼することなく、参照、リプレイ、監査、引用できる。

9. Security Considerations (セキュリティの考慮事項)

9.1 伸長攻撃 (Length Extension Attack) への耐性

SHA-256 などの古典的な Merkle-Damgard 型ハッシュ関数は、secret-prefix MAC に類似する構成で誤用された場合、Length Extension Attack に注意が必要である。Chat-Turn HashChain Governance は、ハッシュペイロード内に秘密鍵を直接含めず、生ハッシュを secret-prefix 認証コードとして利用しない。

したがって、本仕様のハッシュチェーン層は keyless である。完全性は、正準化ペイロードの束縛、前段ハッシュの束縛、および IChatTurnSignatureProvider による任意署名によって提供される。さらに、Merkle-Damgard 型の length-extension misuse class を避けたいデプロイメントでは、ISemanticHasher の参照実装として、スポンジ構造を持つ SHA-3 / Keccak 系の利用を推奨できる。

これは、衝突耐性、正準化の正しさ、署名検証、Provider policy validation が不要になることを意味しない。本仕様が要求するのは、ハッシュチェーン構造そのものが secret-prefix hash pattern に依存しないことである。

9.2 プロンプトインジェクションと汚染波及の遮断

攻撃者が悪意ある命令をチャットターンの Body に混入させた場合、そのターンは有効チェーンへ受理される前に Pre-Inference Governance によって検査されなければならない。拒否または疑わしいターンは、独自の因果メタデータを持つ Quarantine に記録できるが、信頼済み Semantic Context の一部として扱ってはならない。

CTHC 構造は、汚染の局所化を可能にする。受理済みの各ターンは、index、timestamp、canonical payload、previous hash、current hash に束縛されるため、Kernel は汚染された発話がどこで発生し、どの下流ターンがそれに依存したかを特定できる。これにより、quarantine checkpoint を構成し、不信頼な Body content が System Prompt、Kernel 設定、Provider credential、その他の Core Substrate state を変更することを防ぐ。

9.3 Hash Collision Resistance

採用されるハッシュアルゴリズムは、対象とする脅威モデルに対して十分な衝突耐性を持たなければならない。SHA-256 は古典的な実用基準として妥当であり、必要に応じて SHA-3 も使用できる。正準化プロセスは、意味的に異なるターンを同一のハッシュペイロードへ潰してはならない。

9.4 Signature Forgery and Replay Attacks

署名アルゴリズムはデプロイメントポリシーに従って選択される。 H_n に H_{n-1} を取り込むことで、過去の署名済みターンを別チェーンの別位置へ移植する攻撃は検出可能となる。

9.5 Chain-Break Detection

PrevHash が現在の末尾と一致しない場合、チェーンは破断している。Kernel は Fail-Closed で停止し、セキュリティアラートまたはリカバリワークフローを起動しなければならない。

9.6 Cryptographic Agility (暗号アルゴリズムの俊敏性)

AIKernel の HashChain および署名ガバナンスは、特定の暗号アルゴリズムに依存しないよう設計されている。これは意図的なアーキテクチャ判断である。耐量子計算機暗号の標準化は進んでいるが、多くの運用環境では、ネイティブランタイムサポート、サイドチャネル耐性、相互運用性、性能特性がまだ発展途上である。

Provider 抽象化によるアルゴリズム非依存性 ハッシュ計算および署名は、以下の Provider インターフェースにより抽象化される。

- ISemanticHasher
- IChatTurnSignatureProvider

したがって、AIKernel Core は、特定の環境が古典暗号を使うのか耐量子暗号を使うのかを知る必要がない。アルゴリズムの差し替えは、依存性注入およびポリシー設定によって実現できる。

アルゴリズムタグ付きペイロード形式 ROM、VFS、ReplayLog に保存される署名ペイロードは、明示的なアルゴリズムタグを持たなければならない。

<algorithm-id>:<base64-signature>

例:

ed25519:AbCdEf...
ml-dsa-65:ZyXwVu...

これにより、過去署名との互換性、耐量子暗号への段階的移行、Provider ルーティング、将来の Federation が可能になる。

v0.1.0 実装方針 v0.1.0 では、対象プラットフォームおよびライブラリスタックがサポートする場合、Ed25519 や ECDSA P-256 などの古典暗号を実用上の基準として推奨する。これらは成熟しており、広く実装されており、Fail-Closed ガバナンス動作を検証するには十分である。

ML-DSA などの耐量子暗号は、運用成熟度、ランタイムサポート、サイドチャネル対策、デプロイメントポリシーが整った段階で、IChatTurnSignatureProvider の差し替えによって導入すべきである。

したがって、本仕様において Cryptographic Agility は Core invariant である。チェーン形式はアルゴリズム識別子を保持し、Core Governance はハードコードされたアルゴリズムではなく Provider Contract を通じて検証をルーティングしなければならない。

10. Minimal Complete Example（完全な具象実行例）

10.1 シナリオと正準化計算

以下は、3 ターンの統治された対話例である。ハッシュ値は可読性のために短縮された例示値である。実際の実装では、設定された ISemanticHasher の完全な出力を用いなければならない。

Turn 1: User Init

- Input actor: User
- Body: Hello, AIKernel.
- Timestamp: 2026-05-31T10:00:00.000Z
- Previous hash H_0 : 0x00000000
- Canonicalized data C_1 :
actor:User
body:Hello, AIKernel.
timestamp:2026-05-31T10:00:00.000Z
- Computed hash H_1 : $Hash(C_1 || "0x00000000") \rightarrow 0x1619E3AA$

Turn 2: Assistant Response

- Input actor: Assistant
- Body: System initialized.
- Timestamp: 2026-05-31T10:00:02.000Z
- Previous hash H_1 : 0x1619E3AA
- Canonicalized data C_2 :
actor:Assistant
body:System initialized.
timestamp:2026-05-31T10:00:02.000Z
- Computed hash H_2 : $Hash(C_2 || "0x1619E3AA") \rightarrow 0x60192CAF$

Turn 3: User Next Command

- Input actor: User
- Body: Load ROM.
- Timestamp: 2026-05-31T10:00:05.000Z
- Previous hash H_2 : 0x60192CAF
- Canonicalized data C_3 :
actor:User
body:Load ROM.
timestamp:2026-05-31T10:00:05.000Z
- Computed hash H_3 : $Hash(C_3 || "0x60192CAF") \rightarrow 0x07743DFF$

10.2 検証プロセス

ランタイムが履歴の完全性を検証する際、IChatTurnChainVerifier は記録された前段 H_2 (0x60192CAF) と正準化ペイロード C_3 を結合し、次のハッシュを独立に再計算する。その値が記録されている H_3 (0x07743DFF) と一致し、すべてのガバナンスおよび署名ポリシーがターンを受理すれば、Turn 2 から Turn 3 への因果関係は、設定された暗号学的前提の下で有効である。

この手続きを genesis hash から final tail hash まで繰り返すことで、対話履歴全体の連続性を検証できる。過去の正準化ペイロードが少しでも変更されると、そのハッシュが変化し、すべての下流リンクが無効化される。

11. Future Work

将来の拡張には以下が含まれる。

- Phase-3 Trust Layer および自己進化型 Governance との統合
- DAG 型因果構造を用いたマルチエージェントチェーンマージ
- Cross-provider signature federation
- 長期的な鍵ローテーションとアルゴリズム移行
- 署名付き replay boundary manifest
- ハードウェア支援署名および鍵管理
- ランタイムサポートと運用実践が成熟した段階での PQC Provider 評価

12. Non-Claims and Limitations

本仕様は、HashChain によって LLM 出力そのものが決定論的になると主張しない。本仕様が決定論的にするのは、受理済みターン列の検証とリプレイである。

本仕様は、特定の正準化標準を強制しない。ただし、実装は既存の canonical JSON アプローチと整合させてもよい。

本仕様は、v0.1.0 において耐量子暗号の利用を必須としない。代わりに、将来の暗号 Provider を Core semantics を変更せず導入できるよう、Cryptographic Agility を要求する。

本仕様は、Quarantine されたターンを信頼できるものにするわけではない。Quarantine は、不正なターンが有効チェーンへ混入することを防ぐための境界である。

13. Conclusion

Chat-Turn HashChain Governance は、チャット履歴を、因果的に連結され、暗号学的に検証可能で、リプレイ可能な意味的実行記録へ変換する。決定論的正準化、前段ハッシュ結合、任意署名、

Fail-Closed 検証、Quarantine、Replay、Cryptographic Agility を組み合わせることで、AIKernel は Semantic Trajectory の完全性を保護する Core 機構を得る。

本仕様は、チャットターンを一時的な UI メッセージではなく、Semantic Context Operating System における統治された実行ノードとして扱うための最小契約境界を提供する。

References

1. Rundgren, Anders, Bryce Jordan, and Samuel Erdtman. "JSON Canonicalization Scheme (JCS)." RFC 8785, 2020. DOI: 10.17487/RFC8785.
2. National Institute of Standards and Technology. Secure Hash Standard (SHS). FIPS 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.
3. National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. FIPS 202, 2015. DOI: 10.6028/NIST.FIPS.202.
4. Josefsson, Simon, and Ilari Liusvaara. "Edwards-Curve Digital Signature Algorithm (EdDSA)." RFC 8032, 2017. DOI: 10.17487/RFC8032.
5. National Institute of Standards and Technology. Digital Signature Standard (DSS). FIPS 186-5, 2023. DOI: 10.6028/NIST.FIPS.186-5.
6. National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard. FIPS 204, 2024. DOI: 10.6028/NIST.FIPS.204.
7. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model." Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
8. Sogawa, Takuya. "Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture." Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
9. Sogawa, Takuya. "AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems." Zenodo, 2026. DOI: 10.5281/zenodo.20460322.